GUÍA BASICA PDO

¿Qué es PDO?

PDO (PHP Data Objects) es una interfaz de acceso a bases de datos en PHP que proporciona una manera consistente y orientada a objetos de interactuar con múltiples sistemas de bases de datos. Fue introducida a partir de PHP 5.1.

A diferencia de otras extensiones como mysqli, PDO permite escribir código que sea independiente del sistema de gestión de base de datos (como MySQL, PostgreSQL, SQLite, etc.), lo que favorece la portabilidad.

¿Para qué se usa?

PDO se utiliza para:

- Conectarse a bases de datos.
- Ejecutar consultas SQL (SELECT, INSERT, UPDATE, DELETE).
- Preparar y ejecutar sentencias preparadas (prepared statements) para mejorar la seguridad (protección contra inyecciones SQL).
- Obtener resultados en distintos formatos (arrays asociativos, objetos, etc.).

¿Cuándo se usa?

Se recomienda usar PDO cuando:

- Se requiere compatibilidad con múltiples motores de base de datos.
- Se desea aprovechar las ventajas de la programación orientada a objetos.
- Se quiere escribir código más seguro utilizando consultas preparadas.
- Se busca una solución moderna y mantenible.

Característica	PDO	mysqli
Soporte de múltiples DB	Sí (MySQL, PostgreSQL, SQLite, etc.)	No (solo MySQL y MariaDB)
Orientado a objetos	Sí	Sí (también tiene estilo procedural)
Consultas preparadas	Sí	Sí
Portabilidad	Alta	Baja
Manejo de errores	Excepciones (más limpio y controlado)	Códigos de error

Ejemplo de Modelo:

```
<?php
// Clase para manejar la conexión a la base de datos con PDO
class MySQL {
    // Variable para almacenar la conexión PDO
    private $conexion;

    // Método para establecer la conexión
    public function conectar() {
        $host = 'localhost';
        $dbname = 'nombre_bd';
        $usuario = 'root';
        $contrasena = '';

        $dsn = "mysql:host=$host;dbname=$dbname;charset=utf8mb4";

        try {
            $this->conexion = new PDO($dsn, $usuario, $contrasena);
            $this->conexion->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);
        } catch (PDOException $e) {
            die("Error de conexión: " . $e->getMessage());
        }
    }

    // Método para cerrar la conexión
    public function desconectar() {
        $this->conexion = null;
    }
```

```
    // Método para obtener la conexión PDO (para consultas preparadas)
    public function getConexion() {
        return $this->conexion;
    }
}
?>
```

Ejemplo de inserción:

```
<?php
require_once '../model/MySQL.php';

// Crear instancia y conectar
$mysql = new MySQL();
$mysql->conectar();

// Obtener datos del formulario
$nombre = $_POST['nombre'];
$cantidad = $_POST['cantidad'];
$precio = $_POST['precio'];

// Preparar la consulta con parámetros
$sql = "INSERT INTO productos (id, nombre, precio, cantidad) VALUES (NULL,
:nombre, :precio, :cantidad)";

try {
    // Preparar y ejecutar la consulta
    $stmt = $mysql->getConexion()->prepare($sql);
```

```
    $stmt->bindParam(':nombre', $nombre);
    $stmt->bindParam(':precio', $precio);
    $stmt->bindParam(':cantidad', $cantidad);
    $stmt->execute();
} catch (PDOException $e) {
    echo "Error al insertar: " . $e->getMessage();
}

// Desconectar
$mysql->desconectar();
?>
```

UPDATE producto:

```
<?php
require_once '../model/MySQL.php';


if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['id'], $_POST['nombre'], $_POST['precio'], $_POST['stock'])) {
    $id = intval($_POST['id']);
    $nombre = trim($_POST['nombre']);
    $precio = floatval($_POST['precio']);
    $stock = intval($_POST['stock']);


    $mysql = new MySQL();
    $mysql->conectar();


    try {
        $sql = "UPDATE productos
                SET nombre_pro = :nombre, precio_pro = :precio, stock_pro = :stock
                WHERE id_pro = :id";


        $stmt = $mysql->getConexion()->prepare($sql);


        $stmt->bindParam(':nombre', $nombre, PDO::PARAM_STR);
        $stmt->bindParam(':precio', $precio);
        $stmt->bindParam(':stock', $stock, PDO::PARAM_INT);
        $stmt->bindParam(':id', $id, PDO::PARAM_INT);


        if ($stmt->execute()) {
            echo json_encode(['success' => true, 'message' => 'Producto actualizado correctamente']);
```

```
        } else {
            http_response_code(500);
            echo json_encode(['success' => false, 'message' => 'Error al actualizar el producto']);
        }
    } catch (PDOException $e) {
        http_response_code(500);
        echo json_encode(['success' => false, 'message' => 'Error en la consulta: ' . $e->getMessage()]);
    }

    $mysql->desconectar();
} else {
    http_response_code(400);
    echo json_encode(['success' => false, 'message' => 'Datos incompletos']);
}
?>
```

Controlador LOGIN:

```
<?php
require_once '../model/MySQL.php';
session_start();

$mysql = new MySQL();
$mysql->conectar();

if (
    isset($_POST['usuario'], $_POST['password']) &&
    !empty($_POST['usuario']) && !empty($_POST['password'])
) {
    // Sanitizar email y obtener contraseña
    $correo = filter_var(trim($_POST['usuario']), FILTER_SANITIZE_EMAIL);
    $password = $_POST['password'];

    try {
        // Consulta preparada para evitar inyección
        $sql = "SELECT * FROM usuarios WHERE usuario_usu = :correo LIMIT 1";
        $stmt = $mysql->getConexion()->prepare($sql);
        $stmt->bindParam(':correo', $correo, PDO::PARAM_STR);
        $stmt->execute();

        $usuario_data = $stmt->fetch(PDO::FETCH_ASSOC);
```

```
        if ($usuario_data) {
            // Verificar contraseña usando password_verify
            if (password_verify($password, $usuario_data['password_usu'])) {
                // Guardar datos en sesión
                $_SESSION['id'] = $usuario_data['id_usu'];
                $_SESSION['login_time'] = time();

                // Redirigir a dashboard
                header("Location: ../dist/pages/dashboard.php");
                exit();
            } else {
                // Contraseña incorrecta
                header("Location: ../index.html?error=2");
                exit();
            }
        } else {
            // Usuario no encontrado
            header("Location: ../index.html?error=1");
            exit();
        }
    } catch (PDOException $e) {
        error_log("Error en login: " . $e->getMessage());
        header("Location: ../index.html?error=500");
        exit();
    }
} else {
    // Datos incompletos
```

```
    header("Location: ../index.html?error=11");
    exit();
}

$mysql->desconectar();
?>
```

Delete:

```
<?php
header('Content-Type: application/json');

require_once '../model/ConexionPDO.php'; // Asegúrate de tener una clase para PDO

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['id_pro'])) {
    $id = intval($_POST['id_pro']);

    try {
        $db = new ConexionPDO(); // Tu clase debe devolver un objeto PDO
        $pdo = $db->conectar();

        // Verificar si el producto está en alguna venta
        $stmtCheck = $pdo->prepare("SELECT COUNT(*) AS total FROM ventas_has_productos WHERE productos_id_pro = :id");
        $stmtCheck->bindParam(':id', $id, PDO::PARAM_INT);
        $stmtCheck->execute();

        $result = $stmtCheck->fetch(PDO::FETCH_ASSOC);
```

```
        if ($result && $result['total'] > 0) {
            echo json_encode([
                'success' => false,
                'message' => 'Este producto ya fue vendido. No se puede eliminar.'
            ]);
            exit;
        }

        // Eliminar si no está asociado a ninguna venta
        $stmtDelete = $pdo->prepare("DELETE FROM productos WHERE id_pro =
:id");
        $stmtDelete->bindParam(':id', $id, PDO::PARAM_INT);

        if ($stmtDelete->execute()) {
            echo json_encode([
                'success' => true,
                'message' => 'Producto eliminado correctamente.'
            ]);
        } else {
            echo json_encode([
                'success' => false,
                'message' => 'No se pudo eliminar el producto.'
            ]);
        }

    } catch (PDOException $e) {
        echo json_encode([
            'success' => false,
```

```
            'message' => 'Error en la base de datos: ' . $e->getMessage()
        ]);
    }
} else {
    echo json_encode(['success' => false, 'message' => 'ID inválido.']);
}
```

Tipos de datos:

Constante PDO	Tipo de Dato PHP	Descripción
PDO::PARAM_BOOL	bool	Valor booleano (true o false)
PDO::PARAM_NULL	null	Valor NULL
PDO::PARAM_INT	int	Entero (número sin decimales)
PDO::PARAM_STR	string	Cadena de texto
PDO::PARAM_LOB	resource / binarios	Datos binarios u objetos grandes (ej. blobs o archivos)
PDO::PARAM_STMT	PDOStatement	Solo se usa internamente por PDO (no es común en uso general)
PDO::PARAM_INPUT_OUTPUT	int	string (referencia)

Tabla Manejo De Errores:

Método/Constante	Descripción	Ejemplo
PDO::ATTR_ERRMODE	Define cómo PDO maneja los errores. Se puede configurar con los valores: PDO::ERRMODE_SILENT, PDO::ERRMODE_WARNING, o PDO::ERRMODE_EXCEPTION (por defecto es PDO::ERRMODE_SILENT).	PDO::ERRMODE_EXCEPTION es el más común para lanzar excepciones.
try / catch	Usado para capturar excepciones que pueden ocurrir durante la ejecución de consultas.	try { // código PDO } catch (PDOException $e) { echo 'Error: ' . $e->getMessage(); }
getMessage()	Recupera el mensaje de error de una excepción.	echo $e->getMessage();
getCode()	Obtiene el código de error SQL (si está disponible) de la excepción.	echo $e->getCode();
errorInfo()	Devuelve un array con información sobre el error en la última operación de la base de datos. **NO usa excepciones.**	print_r($pdo->errorInfo());
errorCode()	Devuelve el código de error SQL de la última operación (si existe).	echo $pdo->errorCode();
setAttribute()	Se usa para cambiar el modo de manejo de errores durante la ejecución de la conexión.	$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

Tipos de Obtención de Resultados:

Método	Descripción	Ejemplo
fetch()	Recupera **una sola fila** a la vez. Devuelve false cuando no hay más resultados.	$stmt = $pdo->prepare("SELECT * FROM usuarios WHERE id = :id"); $stmt->execute([':id' => 1]); $usuario = $stmt->fetch(PDO::FETCH_ASSOC);
fetchAll()	Recupera **todas las filas** y devuelve un **array**.	$stmt = $pdo->prepare("SELECT * FROM usuarios"); $stmt->execute(); $usuarios = $stmt->fetchAll(PDO::FETCH_ASSOC);
fetchColumn()	Recupera **una sola columna** de la primera fila. Normalmente se usa para contar o recuperar valores específicos.	$stmt = $pdo->prepare("SELECT nombre FROM usuarios WHERE id = :id"); $stmt->execute([':id' => 1]); $nombre = $stmt->fetchColumn();
fetchObject()	Recupera **una fila como objeto** de la clase stdClass o una clase personalizada si se especifica.	$stmt = $pdo->prepare("SELECT * FROM usuarios WHERE id = :id"); $stmt->execute([':id' => 1]); $usuario = $stmt->fetchObject();

Ejemplo procedimientos almacenados:

```
$stmt = $pdo->prepare("CALL obtener_usuario(:id)");

$stmt->execute([':id' => 1]);

$usuario = $stmt->fetch(PDO::FETCH_ASSOC);
```